

Autonomous Library Reshelving

A Planning and Reasoning Project

M.Sc. Artificial Intelligence and Robotics

Vano Mazashvili (1993251)

Planning & Reasoning — A.A. 2025/26



SAPIENZA
UNIVERSITÀ DI ROMA



The Domain

Mobile robots maintain a library by reshelving returned books.

Returned books accumulate on a central *return cart*. Battery-powered robots collect them and redistribute each to a category-matched shelf (fiction, science, history). Navigation consumes battery; robots recharge at *charging docks*.

- **Robots** — battery-powered, mobile across zones
- **Books** — each belongs to one category
- **Shelves** — category-matched destinations
- **Docks** — recharge points on the map

Two formalisms: PDDL (classical STRIPS + numeric ENHSP) and IndiGolog (situation calculus + controllers).

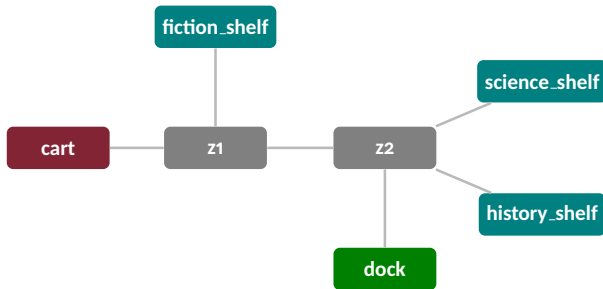
Goal

Every returned book ends up shelved on a shelf of its own category — while the model handles battery economics and dynamic events.



Zone Topology

The IndiGolog component uses a 7-zone graph. Edges are bidirectional; robots navigate one hop at a time. Books start at the cart; each shelf accepts one category; the dock is the recharge point.



cart return cart **gray** junction **teal** shelf (category) **green** charging dock



Table of Contents

1 PDDL

► PDDL

► IndiGolog



1. PDDL

- Domain: predicates & actions
- A key design decision (navigation & battery)
- Three problem instances
- STRIPS heuristic comparison & findings
- Numeric (ENHSP) formulation & findings



STRIPS Domain — Predicates & Actions

1 PDDL

Fluents (change over time)

<code>robot-at(r,z)</code>	robot <code>r</code> in zone <code>z</code>
<code>book-at(b,z)</code>	book <code>b</code> in zone <code>z</code>
<code>carrying(r,b)</code>	<code>r</code> carries book <code>b</code>
<code>shelved(b)</code>	<code>b</code> correctly shelved
<code>battery-low(r)</code>	<code>r</code> 's battery is low
<code>free(r)</code>	<code>r</code> carries nothing

Rigid (static)

<code>path(z1,z2)</code>	zones adjacent
<code>book-category(b,c)</code>	<code>b</code> has category <code>c</code>
<code>shelf-category(z,c)</code>	<code>z</code> accepts <code>c</code>
<code>station-at(z)</code>	<code>z</code> has a dock

Actions

<code>navigate(r,z1,z2)</code>	drive along a path edge
<code>pickup(r,b)</code>	collect a co-located book
<code>shelve(r,b)</code>	place on matching shelf — <i>depletes battery</i>
<code>recharge(r)</code>	restore battery at a dock



Design Decision — Navigation & Battery

1 PDDL

In the STRIPS and IndiGolog models, navigate has no battery precondition.

Requiring a charged battery to move would create **unrecoverable dead-ends**: a robot could strand itself at a shelf with a low battery and no way to reach a dock. So navigation is always permitted along an existing edge. The numeric model is the exception — it *can* safely guard navigate (battery ≥ 10) because its planner reasons about quantities and plans recharges ahead; the boolean models cannot, so they omit the precondition.

Why it matters

Battery as an action precondition can make a goal physically unreachable — the planner reports failure not because no plan exists in principle, but because the model painted the robot into a corner.

How each model handles it

STRIPS — battery is an abstract boolean flag; `shelve` depletes it, `recharge` clears it.

Numeric (ENHSP) — battery is a quantity (0–100); the planner balances delivery against recharging trips.



Three Problem Instances

1 PDDL

Each instance increases in complexity, exercising the planners at different scales.

	Robots	Zones	Books	Categories	Docks
P1	1	5	2	2	1
P2	2	8	4	3	1
P3	2	12	6	4	2

Topology is a connected grid; P3 adds a second dock and a fourth category, pushing the optimal-search configurations to their limits.



STRIPS — Heuristic Comparison (A^*)

1 PDDL

Fast-Downward, A^* with four heuristics. Plan cost = length (unit actions). Expansions shown per problem.

Heuristic	P1 len	P1 exp	P2 len	P2 exp	P3 len	P3 exp
Blind	13	45	27	17,355	44	473,273
$h^{\max}$	13	37	27	15,082	44	447,251
h^{add}	13	16	27	3,893	46*	106,880
h^{FF}	13	26	27	9,496	44	346,822

* h^{add} returns a 46-step plan on P3 — suboptimal (optimum is 44). Detailed next slide.



STRIPS — Key Findings (Problem 3)

1 PDDL

Inadmissibility, observed

h^{add} returned a 46-step plan where Blind, $h^{\max}$ and h^{FF} all found the 44-step optimum. By overestimating, A^* committed early to a non-optimal path. (h^{FF} is also inadmissible but stayed optimal here — luck, not guarantee.)

Informativeness $\neq$ speed

$h^{\max}$ and h^{FF} were *slower* than Blind in wall-clock (2.30 s, 2.36 s vs 1.19 s) despite fewer expansions: on this grid topology, delete-relaxation heuristics are loose, so per-node cost outweighs pruning.

Cost-model artifact

Under unit action costs, dock_2 is never used in P3 — dock_1's central position dominates. The numeric model (differential costs) is where dock_2 could earn its place.



Numeric PDDL — Battery as a Fluent

1 PDDL

Per Prof. Marrella's recommendation, an additional numeric version models battery quantitatively with ENHSP.

Model

battery	numeric 0-100
navigate	-10 batt, +1 cost
shelve	-30 batt, +2 cost
recharge	set 100, +3 cost
metric	minimize total-cost

What it captures

Unlike the STRIPS flag, the numeric model expresses real battery economics: navigation costs energy, and the planner must weigh delivery against recharging trips. This answers a different question — not the shortest plan, but the most energy-efficient one.



ENHSP — Configuration Comparison

1 PDDL

Four configurations across the three problems. Satisficing (sat-) vs. optimal (opt-) search.

Config	P1 cost	P2 cost	P3 cost	P3 time
aibr	20 (subopt)	—	—	—
sat-hadd	15	34	64	0.27 s
opt-hmax	15	31	timeout	>120 s*
opt-hrmax	15	31	timeout	>120 s*

** Optimal configs do not terminate within 120 s on P3, while satisficing solves in 0.27 s.*



Numeric — Key Finding

1 PDDL

Exponential blow-up in optimal numeric planning

On Problem 3, `sat-hadd` and `sat-hff` return plans in 0.27 s, while `opt-hmax` and `opt-hrmax` fail to terminate within 120 s. This is the empirical signature of the cost of admissibility: guaranteeing optimality over a numeric state space is dramatically more expensive than finding a good-enough plan.

STRIPS vs. Numeric — two different questions

STRIPS asks “what is the shortest plan?” — P3 optimum 44 steps in ~ 1.2 s.

Numeric asks “what is the most energy-efficient plan?” — P3 satisficing 52 steps in 0.27 s, with explicit battery accounting the STRIPS model cannot express.



Table of Contents

2 IndiGolog

► PDDL

► IndiGolog



2. IndiGolog

- Situation-calculus formulation (the BAT)
- Fluents, actions, causal laws
- Reasoning tasks: legality & projection
- Controller 1 — basic search
- Controller 2 — reactive (prioritized interrupts)



Situation-Calculus Formulation

2 IndiGolog

The same domain is recast as a **Basic Action Theory** and implemented on the standalone `indigolog_plain` interpreter.

Interpreter

`indigolog_plain` — the vanilla architecture used by the course's elevator examples. It natively supports exogenous actions, `search`, and `prioritized_interrupts`; no deployment engine, devices, or networking needed.

Single robot

The PDDL component already demonstrates multi-robot coordination (P2, P3). IndiGolog focuses on the reasoning tasks and control constructs, so a single agent keeps controller traces legible — a deliberate, justified adaptation.

Boolean battery

Battery is a boolean here, set only by an exogenous drain event and cleared by `recharge`. The numeric treatment lives in the PDDL part, exactly where the recommendation scoped it.



BAT — Fluents, Actions, Causal Laws

2 IndiGolog

Written in indigolog_plain style: prim_fluent, value-equality conditions, causes_val successor-state axioms.

```
% fluents (boolean modelled as value-equality, never bare atoms)
```

```
prim_fluent(robot_loc(R)) :- robot(R).
```

```
prim_fluent(battery_low(R)) :- robot(R).
```

```
% precondition: shelve needs a charged battery and a matching shelf
```

```
poss(shelve(R,B), and(carrying(R)=B,  
                      and(robot_loc(R)=Z,  
                          and(shelf_category(Z,C),  
                              and(book_category(B,C), battery_low(R)=false)))))).
```

```
% causal laws - navigate does NOT deplete battery (see design decision)
```

```
causes_val(navigate(R,_,To), robot_loc(R), To, true).
```

```
causes_val(battery_drained(R), battery_low(R), true, true).
```

```
causes_val(recharge(R), battery_low(R), false, true).
```



Reasoning Tasks — Legality & Projection

2 IndiGolog

1 — Legality

Is a sequence executable from S_0 ?

```
legal_seq([  
  navigate(r1, cart, z1),  
  navigate(r1, z1, fiction_shelf)]).  
-> true
```

```
legal_seq([  
  navigate(r1, cart, z1),  
  pickup(r1, b1)]).  
-> false
```

2 — Projection

Does a fluent hold after a sequence?

```
holds_after(  
  [navigate(r1, cart, z1)],  
  robot_loc(r1)=z1).  
-> true
```

```
holds_after(  
  [navigate(r1, cart, z1)],  
  battery_low(r1)=false).  
-> true
```



Controller 1 — Basic Search

2 IndiGolog

A goal-directed, bounded program wrapped in the IndiGolog search operator finds a complete plan offline. A visited-set in routing forces simple paths.

main(basic). → 19 actions

```
pickup(r1,b1)
navigate(r1, cart, z1)
navigate(r1, z1, fiction_shelf)
shelve(r1, b1)
navigate(r1, fiction_shelf, z1)
navigate(r1, z1, cart)
pickup(r1, b2)
...
shelve(r1, b3)
```

Constructs: search (offline lookahead), pi/ndet (nondeterministic choice), while (loop until all shelved).

Finding: a plan, not the best

The first working version wandered (35 actions). A visited-set forbidding revisits forces acyclic routes → 19 actions. Same lesson as the PDDL h^{add} finding: quality needs structure or an admissible heuristic.



Controller 2 — Reactive (Prioritized Interrupts)

2 IndiGolog

`prioritized_interrupts` react to three exogenous events. Priority arbitrates between them; each produces a genuine behavioural response.

`battery_drained(r)` divert to dock →
recharge → resume

`urgent_request(b)` serve book *b* first
(pre-empt)

`new_return(b)` book reappears at cart →
reselve

`battery_drained(r1)` injected

```
start_interrupts
navigate(r1, cart, z1)
navigate(r1, z1, z2)
navigate(r1, z2, dock)
recharge(r1)          <- reaction
navigate(r1, dock, z2)
...
shelve(r1, b3)
stop_interrupts
```



Cross-Component Synthesis

2 IndiGolog

One domain, three complementary treatments

STRIPS

battery = boolean flag
planning over an abstraction

Numeric

battery = quantity 0-100
energy economics, ENHSP

IndiGolog

battery = reactive event
drive-to-dock & recharge

Depletion differs by design: STRIPS auto-depletes on *shelf*; IndiGolog depletes only via the exogenous *battery_drained* event — the reactive controller needs an external trigger.

Suboptimality, twice

PDDL: h^{add} returns 46 vs the 44 optimum (inadmissibility). IndiGolog: naive search wanders 35 steps until a visited-set forces 19. Same lesson — quality needs an admissible heuristic or a structural constraint.

Reactive vs. planned

The search controller is globally optimal but offline. The reactive controller is responsive but locally greedy — it recharges, then returns to the cart, a visible non-optimal detour. Two paradigms, two trade-offs.



Adaptations Justified at Presentation

2 IndiGolog

Per the project rules, the domain and problem instances stay fixed; component divergences are declared here:

- **Single robot** in IndiGolog vs. up to two in PDDL — focus on reasoning tasks and control constructs, not coordination.
- **Boolean battery** in IndiGolog vs. numeric in PDDL — numeric treatment scoped to the PDDL part, as recommended.
- **Exogenous events** (`urgent_request`, `new_return`, `battery_drained`) have no PDDL counterpart — they exercise `prioritized_interrupts`.

Repository: `github.com/vmazashvili/planning-reasoning-library-reshelving`



Autonomous Library Reshelving

Thank you for listening!
Any questions?